

Setting Up The Java Development Environment

Generated Jul 7, 2026 1:26 PM

What We'll Learn

By the end of this lesson, we can:

- Differentiate JDK, JRE, JVM, source code, and IDE.
- Follow the basic steps for preparing a Java development environment.
- Run a simple Java project or verify that Java is installed.

Words We'll Use

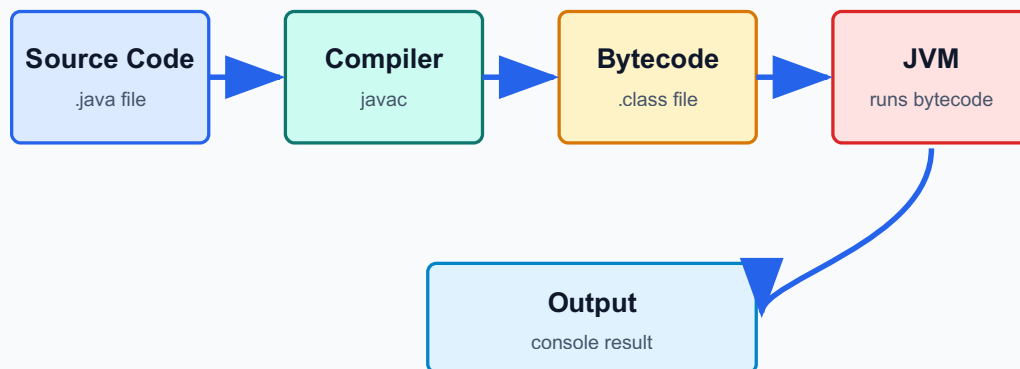
- JDK - Java Development Kit used to build Java programs.
- JRE - Java Runtime Environment used to run Java applications.
- JVM - Java Virtual Machine that executes Java bytecode.
- IDE - software used to write and manage code.
- Compiler - tool that translates source code.

Before We Start

If a carpenter needs tools to build a table, what tools does a programmer need to build a Java program?

Let's Understand

How Java Tools Work Together



Let's work through Lesson 2: Setting Up The Java Development Environment together. Our focus is the tools needed to write, compile, run, and test Java programs. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means:

- Java is a programming language we use to write instructions for a computer.
- A Java program is written as source code, saved in a `.java` file, compiled, then run.
- The JDK gives us the tools for building Java programs; the JVM runs the compiled bytecode.
- An IDE helps us write, organize, run, and debug code in one workspace.
- A beginner Java program usually has a class, a main method, statements, braces, and comments.
- We read code from the outside container toward the inside instructions, then we trace what will display.

How we study it step by step:

- First, we connect the lesson to a familiar situation: If a carpenter needs tools to build a table, what tools does a programmer need to build a Java program?
- Next, we name the important parts so everyone uses the same vocabulary.
- Then, we study JDK, JRE, JVM and ask: what does it do, where do we see it, and how can we check it?
- Then, we study IDE setup and ask: what does it do, where do we see it, and how can we check it?
- Then, we study Checking Java version and ask: what does it do, where do we see it, and how can we check it?
- Then, we study Creating a first project and ask: what does it do, where do we see it, and how can we check it?
- After that, we compare a correct example with a common wrong version.

- We trace the example slowly before running, drawing, or submitting anything.
- Finally, we explain the result in our own words so the activity is not just copying.

Rules and patterns we should remember:

- Rule: Java is case-sensitive, so `System` and `system` are not the same. Example:
`System.out.println("Hi");` works, but `system.out.println("Hi");` causes an error.
 Check: Which word must start with a capital S?
- Rule: Text output must be inside double quotation marks. Example:
`System.out.println("Hello, Java!");` displays Hello, Java! Check: What text will appear on the screen?
- Rule: Many Java statements end with a semicolon. Example: `int score = 90;` is complete, but `int score = 90` is missing its ending mark. Check: What symbol should we add at the end?
- Rule: Braces `{` and `}` must be paired because they show where a block starts and ends.
 Example: `public class Demo { public static void main(String[] args) { } }` has matching class and main braces. Check: How many opening and closing braces can we count?
- Rule: Comments explain code for humans and are ignored when the program runs. Example: `// This line prints a greeting helps us understand the next statement.` Check: Will a comment display in the console?

Common mistakes we will watch for:

- Forgetting capitalization in Java keywords or class names.
- Missing quotation marks, semicolons, or closing braces.
- Thinking comments run as code.
- Changing code without predicting the output first.
- Submitting a screenshot without explaining what happened.

Mini-check before practice:

- What part of the Java program is the container?
- Where does the program start running?
- What line displays output?
- What syntax rule should we check before running?

When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: Tool Role Sort Type: individual Time: 8 minutes Instruction: Sort each Java tool by what it does. Student task:

- JDK - builds, runs, or writes?
- IDE - builds, runs, or writes?
- Compiler - translates or displays?
- JVM - edits code or runs bytecode?

Output: Submit a tool-role table with four correct rows.

Activity 2: Setup Checklist Type: hands-on coding Time: 12 minutes Instruction: Check whether the computer is ready for Java. Student task:

- Open the IDE or terminal.
- Find the Java version or IDE welcome screen.
- Record what tool opened successfully.
- Write one issue if something did not open.

Output: Submit a setup checklist and screenshot or written evidence.

Activity 3: First Project Test Run Type: hands-on coding Time: 15 minutes Instruction: Create a basic Java project and run a simple output program. Student task:

- Create a new Java file.
- Type the sample program.
- Run the program.
- Circle or copy the console output.

Code / model:

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, CC 102!");  
    }  
}
```

Output: Submit the output and one sentence explaining what ran successfully.

Activity 4: Troubleshooting Log Type: pair Time: 8 minutes Instruction: Practice reading setup or run problems calmly. Student task:

- Write one problem that could happen during setup.
- Write one possible cause.
- Write one possible fix.
- Ask a partner if the fix is clear.

Output: Submit one problem-cause-fix row.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Check tool understanding. Student task:

- Which tool helps us write code?
- Which tool runs bytecode?
- Which command or screen proved Java worked?

Output: Submit three short answers.

Now We Apply

Now we apply it through these tasks:

1. Create a setup checklist for a classmate.
2. Run the Hello World sample and write what happened.
3. Label a diagram showing source code, compiler, JVM, and output.

Challenge Task

Produce a Java Setup Confirmation Sheet with tool names, installation/status notes, a test-run result, and one troubleshooting tip.

How Our Work Will Be Checked

Criteria - 20 points total

- Correctness (5): Output or answer follows the required concept and instructions.
- Completeness (4): All required parts are present.
- Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order.
- Explanation (3): Student can explain what the work does and why it is correct.
- Neatness/readability (2): Work is easy to read, label, and check.
- Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect:

- What concept from this lesson is clearest to you now?
- What mistake did you notice or correct during practice?
- Which activity helped you understand the lesson best?
- How can this skill help you design or write a Java program?
- What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning:

- Students identify JDK and IDE roles.
- A successful run displays simple output.
- Setup checklist is ordered and practical.
- Completed guided-practice work with visible corrections or annotations.
- A short explanation connecting the output to the lesson target.

Student Activities

Java Tools Setup Check: identify whether JDK and IDE are installed. Version Check: record the Java version or IDE welcome screen. Tool Match: match JDK, IDE, compiler, and output window. First Project Test Run: create a new Java project and run a sample program. Screenshot Evidence: capture proof that Java ran successfully. Troubleshooting Log: list one setup issue and solution. Create a setup checklist for a classmate. Run the Hello World sample and write what happened. Label a diagram showing source code, compiler, JVM, and output.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.